Emerging Training Algorithms for Large Language Models: A Technical Survey of Reinforcement Learning Approaches

From RLHF to GRPO: Foundations, Methods, and Perspectives

Wozouba Fahd Mathis BILLA

National Scientific High School of Bobo-Dioulasso

Bobo-Dioulasso, Burkina Faso

wmathisbilla@gmail.com

December 2025

Abstract

The alignment and capability of large language models (LLMs) critically depend on the training algorithms used beyond standard supervised pretraining. Reinforcement Learning from Human Feedback (RLHF) first demonstrated that reward signals derived from human preferences can substantially improve model behavior, launching a wave of research into more efficient, stable, and scalable training paradigms. This paper provides a rigorous technical survey of the most significant emerging training algorithms for LLMs, spanning classical policy gradient methods (REINFORCE, PPO), preference optimization approaches (DPO, IPO, KTO), and recent group-based methods (GRPO). For each algorithm, we derive the objective from first principles, discuss its mathematical properties, analyze failure modes, and provide pseudocode for practical implementation. We further compare the methods along axes of computational cost, stability, sample efficiency, and alignment quality, and discuss open theoretical questions.

Keywords: reinforcement learning from human feedback, RLHF, proximal policy optimization, direct preference optimization, GRPO, LLM alignment, policy gradient, reward modeling.

Contents

1 Introduction

Training a large language model involves at least two distinct phases. In the first phase, *pretraining*, the model learns to predict tokens from a massive corpus via maximum likelihood estimation (MLE). While pretraining equips the model with broad linguistic and factual competence, it does not inherently align model outputs with human intent, values, or safety constraints. The second phase, *post-training*, addresses this gap through a family of algorithms that fine-tune the model using preference signals, reward models, or direct feedback.

The seminal work of Christiano et al. [1] introduced the RLHF paradigm: train a reward model from human pairwise comparisons, then optimize the language model policy against this reward using reinforcement learning. InstructGPT [2] scaled this approach to GPT-class models, demonstrating dramatic improvements in instruction-following and safety. Since then, the field has produced a rich ecosystem of training algorithms that address the limitations of the original RLHF pipeline.

This survey is organized as follows. Section 2 establishes the mathematical framework: Markov decision processes, policy gradient theorems, and KL-divergence regularization. Section 3 covers the RLHF pipeline in detail. Section 4 analyzes Proximal Policy Optimization (PPO) as applied to LLMs. Section 5 derives Direct Preference Optimization (DPO) and its variants. Section 6 covers the recent Group Relative Policy Optimization (GRPO). Section 7 compares all methods. Section 8 discusses open problems. Section 9 concludes.

2 Mathematical Background

2.1 Language Models as Policies

Let $\mathcal{V}$ denote a vocabulary. A prompt $x \in \mathcal{V}^*$ and a response $y \in \mathcal{V}^*$ are sequences of tokens. A language model π_θ defines a conditional distribution over responses:

$$\pi_\theta(y \mid x) \;=\; \prod_{t=1}^{|y|} \pi_\theta(y_t \mid x, y_{<t}).$$

We view π_θ as a *policy* in a bandit-like reinforcement learning setting: given prompt x (the state), the policy generates response y (the action), and receives a scalar reward $r(x, y)$.

2.2 The KL-Regularized Objective

A central concern in LLM post-training is preventing the policy from drifting too far from the reference model π_{ref} (typically the supervised fine-tuned model). The standard *KL-regularized* optimization problem is:

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] \;-\; \beta\,\mathrm{KL}\big[\pi_\theta(\cdot\mid x)\;\|\;\pi_{\text{ref}}(\cdot\mid x)\big], \tag{1}$$

where $\beta > 0$ is the regularization coefficient and $\mathrm{KL}[p\|q] = \mathbb{E}_p[\log(p/q)]$.

Proposition 2.1 (Optimal Policy)**.** *The unique solution to* (1) *is:*

$$\pi^*(y\mid x) \;=\; \frac{1}{Z(x)}\,\pi_{\text{ref}}(y\mid x)\,\exp\!\left(\frac{r(x,y)}{\beta}\right), \tag{2}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y\mid x)\exp(r(x,y)/\beta)$ *is the partition function.*

Proof. Taking the functional derivative of (1) with respect to $\pi_\theta(y\mid x)$ and setting it to zero:

$$r(x,y) - \beta\big(\log\pi_\theta(y\mid x) - \log\pi_{\text{ref}}(y\mid x) + 1\big) = 0,$$

which gives $\log\pi_\theta(y\mid x) = \frac{1}{\beta}r(x,y) + \log\pi_{\text{ref}}(y\mid x) - 1$. Normalizing yields (2). □

2.3 Policy Gradient Theorem

For a general reward $r(x,y)$, the gradient of the expected reward with respect to policy parameters θ is:

$$\nabla_\theta \mathbb{E}_{y\sim\pi_\theta}[r(x,y)] \;=\; \mathbb{E}_{y\sim\pi_\theta}[r(x,y)\,\nabla_\theta\log\pi_\theta(y\mid x)]\,. \tag{3}$$

This is the *policy gradient theorem* [9]. In practice, (3) is estimated via Monte Carlo sampling, yielding the REINFORCE estimator:

$$\widehat{\nabla}_\theta \;=\; \frac{1}{B}\sum_{i=1}^{B} r(x_i,y_i)\,\nabla_\theta\log\pi_\theta(y_i\mid x_i), \quad y_i\sim\pi_\theta(\cdot\mid x_i). \tag{4}$$

The estimator (4) is unbiased but has high variance, which motivates the use of *baselines* and more sophisticated algorithms.

2.4 Reward Modeling and the Bradley–Terry Model

Given a dataset of human preference pairs $\mathcal{D} = \{(x, y^+, y^-)\}$ where y^+ is preferred over y^-, the reward model r_ϕ is trained under the Bradley–Terry model [10]:

$$P(y^+ \succ y^- \mid x) \;=\; \sigma\big(r_\phi(x,y^+) - r_\phi(x,y^-)\big), \tag{5}$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ is the sigmoid function. The reward model loss is:

$$\mathcal{L}_{\text{RM}}(\phi) \;=\; -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}}\big[\log \sigma\big(r_\phi(x,y^+) - r_\phi(x,y^-)\big)\big]\,. \tag{6}$$

3 Reinforcement Learning from Human Feedback (RLHF)

3.1 Pipeline Overview

The RLHF pipeline [1, 2] consists of three stages:

Stage 1. Supervised Fine-Tuning (SFT). Train a language model π_{SFT} on a curated dataset of high-quality (prompt, response) pairs via MLE:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{SFT}}}\left[\log \pi_\theta(y \mid x)\right].$$

Stage 2. Reward Model Training. Sample responses from π_{SFT}, collect human preference labels, and train r_ϕ via (6).

Stage 3. RL Fine-Tuning. Optimize π_θ against r_ϕ subject to KL regularization from $\pi_{\text{ref}} = \pi_{\text{SFT}}$:

$$\max_\theta \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta}\left[r_\phi(x,y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}\right].$$

3.2 Limitations of Vanilla RLHF

Despite its success, the standard RLHF pipeline suffers from several well-documented issues:

- **Reward hacking.** The policy may exploit spurious patterns in r_ϕ that do not reflect true human preferences [11].
- **Training instability.** RL optimization over language models is notoriously unstable due to the high-dimensional, discrete action space.
- **Computational cost.** Stage 3 requires maintaining four models simultaneously: π_θ, π_{ref}, r_ϕ, and a value model V_ψ.
- **Distribution shift.** The reward model is trained on π_{SFT} outputs but evaluated on π_θ outputs, which diverge over training.

4 Proximal Policy Optimization (PPO)

4.1 Motivation and Objective

Proximal Policy Optimization [3] addresses the instability of vanilla policy gradient by constraining the policy update at each step. Let $\rho_t(\theta)$ denote the probability ratio:

$$\rho_t(\theta) \;=\; \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}.$$

The PPO-Clip objective is:

$$\mathcal{L}^{\text{PPO}}(\theta) \;=\; \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\,\hat{A}_t,\;\; \text{clip}(\rho_t(\theta),\,1-\varepsilon,\,1+\varepsilon)\,\hat{A}_t\Big)\Big], \tag{7}$$

where $\varepsilon \in (0,1)$ (typically 0.1–0.2) is the clipping threshold and $\hat{A}_t$ is an estimate of the *advantage function*:

$$\hat{A}_t \;=\; r(x,y) - V_\psi(x, y_{<t}).$$

4.2 Generalized Advantage Estimation

The advantage is estimated via *Generalized Advantage Estimation* (GAE) [4]:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} \;=\; \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t), \tag{8}$$

where $\gamma \in [0,1]$ is the discount factor and $\lambda \in [0,1]$ controls the bias-variance trade-off. In the token-level MDP formulation for LLMs, the reward r_t is nonzero only at the final token (end of sequence).

4.3 PPO for LLMs: Full Objective

The complete PPO objective for LLM alignment combines the policy loss, a value function loss, and an entropy bonus:

$$\mathcal{L}^{\text{total}}(\theta,\psi) \;=\; -\mathcal{L}^{\text{PPO}}(\theta) + c_1\mathcal{L}^{\text{VF}}(\psi) - c_2 H[\pi_\theta(\cdot \mid x)], \tag{9}$$

where $\mathcal{L}^{\text{VF}}(\psi) = \mathbb{E}_t[(V_\psi(s_t) - V_t^{\text{targ}})^2]$, H is the entropy, and $c_1, c_2 > 0$ are coefficients.

Algorithm 1 PPO for LLM Alignment

Require: SFT model π_{ref}, reward model r_ϕ, prompt dataset $\mathcal{D}$, clip ε, KL coefficient β, steps T

Initialize $\pi_\theta \leftarrow \pi_{\text{ref}}$, value model V_ψ
for $t = 1$ to T **do**
  Sample batch $\{x_i\} \sim \mathcal{D}$
  Generate responses $y_i \sim \pi_\theta(\cdot \mid x_i)$ ▷ Rollout
  Compute rewards $\tilde{r}_i = r_\phi(x_i, y_i) - \beta \log \frac{\pi_\theta(y_i|x_i)}{\pi_{\text{ref}}(y_i|x_i)}$
  Compute advantages $\hat{A}_i$ via GAE (8)
  for each mini-batch epoch **do**
    Compute $\rho_t = \pi_\theta / \pi_{\theta_{\text{old}}}$
    Compute clipped loss $\mathcal{L}^{\text{PPO}}$ via (7)
    Update θ via $\nabla_\theta \mathcal{L}^{\text{total}}$
    Update ψ via $\nabla_\psi \mathcal{L}^{\text{VF}}$
  end for
  $\theta_{\text{old}} \leftarrow \theta$
end for
return π_θ

5 Direct Preference Optimization (DPO) and Variants

5.1 DPO: Derivation

Direct Preference Optimization [5] eliminates the need for an explicit reward model by reparameterizing the reward in terms of the policy. From (2), we can express the reward as:

$$r^*(x, y) \;=\; \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{10}$$

Substituting (10) into the Bradley–Terry model (5) and noting that $Z(x)$ cancels:

$$P(y^+ \succ y^- \mid x) \;=\; \sigma\!\left(\beta \log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \beta \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}\right). \tag{11}$$

The DPO loss is thus the negative log-likelihood of (11):

$$\boxed{\mathcal{L}^{\text{DPO}}(\theta) \;=\; -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}}\left[\log \sigma\!\left(\beta \log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \beta \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}\right)\right].} \tag{12}$$

Remark 5.1. DPO requires no reward model, no value model, no rollout generation, and no RL loop. It reduces LLM alignment to a simple supervised classification problem over preference pairs, with memory and compute requirements similar to standard SFT.

Algorithm 2 Direct Preference Optimization (DPO)

Require: Reference model π_{ref}, preference dataset $\mathcal{D} = \{(x_i, y_i^+, y_i^-)\}$, $\beta > 0$
Initialize $\pi_\theta \leftarrow \pi_{\text{ref}}$
for each mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
  for (x, y^+, y^-) in $\mathcal{B}$ **do**
    $\ell^+ \leftarrow \log \pi_\theta(y^+ \mid x) - \log \pi_{\text{ref}}(y^+ \mid x)$
    $\ell^- \leftarrow \log \pi_\theta(y^- \mid x) - \log \pi_{\text{ref}}(y^- \mid x)$
    $\mathcal{L}_i \leftarrow -\log \sigma(\beta(\ell^+ - \ell^-))$
  end for
  Update $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_i \mathcal{L}_i$
end for
return π_θ

5.2 Identity Preference Optimization (IPO)

DPO can overfit when the policy assigns near-zero probability to dispreferred responses. IPO [6] addresses this by replacing the log-sigmoid loss with a squared loss on the implicit reward difference:

$$\mathcal{L}^{\text{IPO}}(\theta) = \mathbb{E}_{(x,y^+,y^-)}\left[\left(\log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)} - \frac{1}{2\beta}\right)^2\right]. \tag{13}$$

The target margin $1/(2\beta)$ acts as a regularizer preventing the implicit reward gap from growing unboundedly.

5.3 Kahneman–Tversky Optimization (KTO)

KTO [7] replaces the pairwise preference assumption with a *prospect-theoretic* model inspired by Kahneman and Tversky's loss aversion framework. Given unpaired data (x, y, z) where $z \in \{0, 1\}$ indicates whether y is desirable:

$$\mathcal{L}^{\text{KTO}}(\theta) = \mathbb{E}_{(x,y,z)}\Big[z \cdot \lambda_D \cdot \big(1 - \sigma(r_\theta(x, y) - z_{\text{ref}})\big) + (1 - z) \cdot \lambda_U \cdot \sigma(r_\theta(x, y) - z_{\text{ref}})\Big], \tag{14}$$

where $r_\theta(x, y) = \beta \log \pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x)$, $z_{\text{ref}} = \mathbb{E}[\text{KL}[\pi_\theta \| \pi_{\text{ref}}]]$, and $\lambda_D, \lambda_U > 0$ are asymmetric loss aversion weights.

6 Group Relative Policy Optimization (GRPO)

6.1 Motivation

GRPO [8] was introduced in the context of mathematical reasoning (DeepSeekMath) and subsequently adopted in DeepSeek-R1. It addresses a key inefficiency in PPO: the value

model V_ψ, which has the same size as the policy, doubles the memory footprint and introduces estimation error in the advantage computation.

6.2 Group-Based Advantage Estimation

Instead of using a learned value baseline, GRPO estimates the advantage by comparing a response against a *group* of G responses sampled from the same prompt:

$$\hat{A}_{i,g} \;=\; \frac{r(x_i, y_{i,g}) - \mu_i}{\sigma_i + \epsilon}, \quad \text{where } \mu_i = \frac{1}{G}\sum_{g=1}^{G} r(x_i, y_{i,g}), \;\; \sigma_i^2 = \frac{1}{G}\sum_{g=1}^{G}(r(x_i, y_{i,g}) - \mu_i)^2. \tag{15}$$

This is a *group-normalized* advantage: each response is evaluated relative to the mean and standard deviation of the group, making the signal independent of the reward scale.

6.3 GRPO Objective

The GRPO training objective combines the clipped policy ratio with the group-normalized advantage and a KL penalty:

$$\mathcal{L}^{\text{GRPO}}(\theta) \;=\; -\frac{1}{G}\sum_{g=1}^{G}\Big[\min\Big(\rho_g(\theta)\,\hat{A}_{i,g},\;\; \text{clip}(\rho_g(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_{i,g}\Big) - \beta\,\text{KL}[\pi_\theta \| \pi_{\text{ref}}]\Big], \tag{16}$$

where $\rho_g(\theta) = \pi_\theta(y_{i,g} \mid x_i)/\pi_{\theta_{\text{old}}}(y_{i,g} \mid x_i)$.

The KL term is computed in closed form using the reference policy, avoiding the need for additional rollouts:

$$\text{KL}[\pi_\theta \| \pi_{\text{ref}}] \;=\; \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - \log\frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - 1. \tag{17}$$

Remark 6.1. GRPO eliminates the value model entirely. The memory savings are substantial: for a 7B-parameter model, removing V_ψ frees roughly 14 GB of GPU memory (at bf16 precision), enabling larger batch sizes or longer sequences.

7 Comparative Analysis

7.1 Theoretical Properties

Table 1 summarizes the key theoretical properties of each algorithm along five axes: whether an explicit reward model is required, whether a value model is maintained during

Algorithm 3 Group Relative Policy Optimization (GRPO)

Require: Reference model π_{ref}, reward function r, prompt dataset $\mathcal{D}$, group size G, clip ε, KL coefficient β

Initialize $\pi_\theta \leftarrow \pi_{\text{ref}}$
for each iteration **do**
  Sample batch $\{x_i\} \sim \mathcal{D}$
  for each x_i **do**
    Sample group $\{y_{i,1}, \ldots, y_{i,G}\} \sim \pi_\theta(\cdot \mid x_i)$
    Compute rewards $\{r_{i,g} = r(x_i, y_{i,g})\}_{g=1}^{G}$
    Compute $\mu_i \leftarrow \frac{1}{G}\sum_g r_{i,g}$, $\sigma_i \leftarrow \sqrt{\frac{1}{G}\sum_g (r_{i,g} - \mu_i)^2}$
    Compute advantages $\hat{A}_{i,g} \leftarrow (r_{i,g} - \mu_i)/(\sigma_i + \epsilon)$
  end for
  $\theta_{\text{old}} \leftarrow \theta$
  for each mini-batch epoch **do**
    Compute $\rho_g = \pi_\theta(y_{i,g} \mid x_i)/\pi_{\theta_{\text{old}}}(y_{i,g} \mid x_i)$
    Compute $\mathcal{L}^{\text{GRPO}}$ via (16)
    Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}^{\text{GRPO}}$
  end for
end for
return π_θ

Table 1: Comparison of LLM training algorithms.

Algorithm	Year	RM needed	Value model	Rollouts	Data type	Stability
REINFORCE	1992	Yes	No	Yes	Prompt	Low
PPO	2017	Yes	Yes	Yes	Prompt	Medium
DPO	2023	No	No	No	Pairs	High
IPO	2023	No	No	No	Pairs	High
KTO	2023	No	No	No	Unpaired	High
GRPO	2024	Yes	No	Yes	Prompt	Medium-High

training, whether on-policy rollouts are needed, the type of training data, and empirical training stability.

Several theoretical distinctions are worth emphasizing:

- **On-policy vs. off-policy.** PPO and GRPO are *on-policy*: they generate fresh rollouts at each iteration, ensuring that gradient estimates are computed under the current policy distribution. DPO, IPO, and KTO are *off-policy*: they optimize over a fixed preference dataset collected under a different (usually SFT) policy. This creates a distribution shift between the dataset and the policy being trained, which can degrade performance when π_θ diverges significantly from π_{ref}.

- **Implicit vs. explicit reward.** PPO and GRPO rely on an explicit, separately trained reward model r_ϕ. DPO and its variants implicitly define the reward through the policy ratio $\hat{r}_\theta(x, y) = \beta \log \pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x)$, bypassing reward model training but

inheriting its own approximation errors.

- **Optimality guarantees.** Under the Bradley–Terry model and infinite data, DPO provably recovers the same optimal policy as RLHF [5]. In practice, finite-sample effects and distribution shift can cause DPO to underperform PPO on complex reasoning tasks where on-policy exploration is essential [8].
- **Stability.** Offline methods (DPO, IPO, KTO) are inherently more stable since they reduce to supervised learning with no RL feedback loop. PPO is susceptible to entropy collapse and reward hacking; GRPO mitigates some of these issues by eliminating the value model bias but retains the instabilities of on-policy training.

7.2 Computational Cost

Let M denote the number of parameters of the base model. The approximate memory requirements (in units of M parameters) during training are:

- **PPO:** $\approx 4M$ (policy π_θ, reference π_{ref}, reward model r_ϕ, value model V_ψ) plus optimizer states.
- **DPO/IPO/KTO:** $\approx 2M$ (policy π_θ, reference π_{ref}) plus optimizer states.
- **GRPO:** $\approx 3M$ (policy π_θ, reference π_{ref}, reward model r_ϕ) plus optimizer states. No value model.

7.3 Gradient Analysis

The gradient of the DPO loss with respect to θ reveals its implicit reward shaping behavior:

$$\nabla_\theta \mathcal{L}^{\text{DPO}} = -\beta\, \mathbb{E}_{(x,y^+,y^-)}\Big[\sigma\big(\hat{r}_\theta(x,y^-) - \hat{r}_\theta(x,y^+)\big) \cdot \big(\nabla_\theta \log \pi_\theta(y^+ \mid x) - \nabla_\theta \log \pi_\theta(y^- \mid x)\big)\Big], \tag{18}$$

where $\hat{r}_\theta(x,y) = \beta \log \pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x)$ is the implicit reward. The factor $\sigma(\hat{r}_\theta(y^-) - \hat{r}_\theta(y^+))$ acts as an adaptive weight: updates are larger when the model incorrectly prefers y^- over y^+, and vanish when the preference is correctly ordered.

8 Open Problems and Future Directions

(1) Reward hacking and overoptimization. As π_θ is optimized against a fixed r_ϕ, the policy inevitably exploits reward model errors [12]. Formal bounds on the overoptimization rate as a function of KL distance remain an open theoretical problem.

(2) Theoretical guarantees for DPO. While DPO is empirically effective, its convergence properties and relationship to the original RLHF objective under distribution shift are not fully understood. In particular, DPO optimizes over the offline dataset distribution, whereas the true objective requires on-policy evaluation.

(3) Optimal group size in GRPO. The advantage estimator (15) has a bias-variance trade-off controlled by G. Theoretical analysis of the optimal G as a function of reward variance and batch size is lacking.

(4) Multi-step reasoning and process reward models. For complex tasks (mathematical reasoning, code generation), outcome-based rewards are insufficient. *Process Reward Models* (PRMs) [13] assign rewards at intermediate reasoning steps, but integrating PRMs with GRPO or DPO raises new theoretical challenges.

(5) Constitutional and self-play methods. Recent work on self-play fine-tuning [15] and constitutional AI [14] suggests that LLMs can generate their own preference data without human annotation. Formalizing the convergence and alignment guarantees of such approaches is an active frontier.

(6) Scaling laws for alignment. Empirical scaling laws for pretraining [16] are well-established, but analogous laws for post-training algorithms—relating compute, data size, and alignment quality—are not yet understood.

9 Conclusion

This survey has presented a rigorous technical treatment of the major emerging training algorithms for large language model alignment. Starting from the KL-regularized reinforcement learning objective and the policy gradient theorem, we derived RLHF, PPO, DPO, IPO, KTO, and GRPO from first principles, highlighting their mathematical relationships and practical trade-offs.

The central tension in this landscape is between *online* methods (PPO, GRPO), which require expensive rollout generation but operate on-policy, and *offline* methods (DPO, IPO, KTO), which are computationally efficient but optimize over static datasets and may suffer from distribution shift. Hybrid approaches that combine the efficiency of offline optimization with the correctness of on-policy training represent a compelling direction for future research.

The rapid pace of progress in this area—with major algorithmic contributions appearing in 2023–2024—suggests that the theoretical foundations of LLM alignment training are still far from settled. We hope this survey provides a useful mathematical reference for researchers entering this field.

References

[1] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Proc. NeurIPS*, 2017, pp. 4299–4307.

[2] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. NeurIPS*, 2022.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[4] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[5] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. NeurIPS*, 2023.

[6] M. G. Azar, M. Rowland, B. Piot, D. Guo, D. Calandriello, M. Valko, and R. Munos, "A general theoretical paradigm to understand learning from human feedback," *arXiv preprint arXiv:2310.12036*, 2023.

[7] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, "KTO: Model alignment as prospect theoretic optimization," *arXiv preprint arXiv:2402.01306*, 2023.

[8] Z. Shao et al., "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[9] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. NeurIPS*, 1999, pp. 1057–1063.

[10] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. The method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.

[11] J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger, "Defining and characterizing reward hacking," in *Proc. NeurIPS*, 2022.

[12] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," in *Proc. ICML*, 2023.

[13] H. Lightman et al., "Let's verify step by step," *arXiv preprint arXiv:2305.20050*, 2023.

[14] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[15] Z. Chen et al., "Self-play fine-tuning converts weak language models to strong language models," in *Proc. ICML*, 2024.

[16] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.